

ui.row 全面详细阐述

在 NiceGUI 框架中，`ui.row` 是核心布局组件之一，用于创建水平排列子元素的容器，通过灵活的配置和丰富的方法，支持复杂的界面布局需求。以下从核心定义、初始化参数、属性、方法、使用示例及注意事项等方面进行全面解析。

一、核心定义

`ui.row` 是一个水平布局容器，其核心作用是将子元素按行排列（默认支持自动换行），适用于需要横向组织的界面元素（如按钮组、标签栏、数据卡片等）。它继承自 NiceGUI 的基础 `Element` 类，因此具备元素通用的属性和方法，同时扩展了水平布局专属的配置项。

二、初始化参数

创建 `ui.row` 时可通过参数配置布局行为，核心参数如下（均为可选参数）：

参数名	类型	说明	默认值
<code>wrap</code>	<code>bool</code>	是否自动换行：当子元素总宽度超过容器宽度时，是否折行显示子元素	<code>True</code>
<code>align_items</code>	<code>str</code> （可选值： <code>"start"</code> 、 <code>"end"</code> 、 <code>"center"</code> 、 <code>"baseline"</code> 、 <code>"stretch"</code> ）	子元素在垂直方向的对齐方式： - <code>"start"</code> ：顶部对齐 - <code>"end"</code> ：底部对齐 - <code>"center"</code> ：垂直居中 - <code>"baseline"</code> ：按基线对齐（文字对齐常用） - <code>"stretch"</code> ：子元素高度拉伸至与容器一致	<code>None</code>

参数使用示例

```
from nicegui import ui

# 1. 默认配置（自动换行，垂直对齐默认）
with ui.row():
    ui.label("默认换行")
    ui.button("按钮1")
    ui.button("按钮2")

# 2. 禁用换行（子元素溢出时不折行）
with ui.row(wrap=False):
    ui.label("禁用换行，溢出可能横向滚动")
    ui.button("按钮A")
    ui.button("按钮B")
    ui.button("按钮C")

# 3. 垂直居中对齐
with ui.row(align_items="center"):
    ui.label("垂直居中")
    ui.button("按钮X")
```

```
ui.icon("star")

ui.run()
```

三、核心属性

`ui.row` 继承自 `Element` 类，拥有以下常用属性（部分为只读，部分支持动态修改）：

属性名	类型	说明	新增版本	
<code>classes</code>	<code>Classes[Self]</code>	元素的 HTML 类名，用于通过 Tailwind/Quasar 样式自定义外观和布局	-	
<code>client</code>	<code>Client</code>	元素所属的客户端实例（只读，用于多客户端场景识别）	-	
<code>html_id</code>	<code>str</code>	元素在 HTML DOM 中的唯一 ID，用于直接操作 DOM 或关联样式	2.16.0	
<code>is_deleted</code>	<code>bool</code>	元素是否已被删除（只读，用于状态判断）	-	
<code>is_ignoring_events</code>	<code>bool</code>	元素是否正在忽略事件（只读，用于事件控制）	-	
<code>parent_slot</code>	<code>`Slot`</code>	<code>None`</code>	元素的父插槽（可设置，用于复杂组件的插槽嵌套）	-
<code>props</code>	<code>Props[Self]</code>	元素的 Quasar props，用于扩展组件功能（如设置尺寸、阴影等）	-	
<code>style</code>	<code>Style[Self]</code>	元素的内联 CSS 样式，用于精细化样式控制	-	
<code>visible</code>	<code>BindableProperty</code>	元素的可见性（支持数据绑定，可动态切换显示 / 隐藏）	-	

属性使用示例

```
from nicegui import ui

# 1. 自定义样式（设置间距、背景色）
row = ui.row(style="gap: 16px; background-color: #f5f5f5; padding: 8px;")
with row:
    ui.label("自定义样式的行")
    ui.button("点击")
```

```
# 2. 设置 HTML ID 和 Classes (结合 Tailwind 样式)
ui.row(html_id="my-row", classes="rounded-lg p-4 shadow-sm").with_content(
    ui.label("带 ID 和 Tailwind 类的行")
)

# 3. 动态控制可见性
row3 = ui.row(visible=False)
with row3:
    ui.label("默认隐藏的行")

# 点击按钮显示
ui.button("显示行3", on_click=lambda: row3.set_visibility(True))

ui.run()
```

四、核心方法

`ui.row` 提供了丰富的方法用于动态操作元素、绑定数据、事件处理等，以下是常用方法分类详解：

（一）布局与子元素操作

方法名	签名	说明		
<code>clear()</code>	<code>() -> None</code>	移除所有子元素 (清空行容器)		
<code>remove()</code>	<code>`(element: Element</code> <code>int) -> None`</code>	移除指定子元素： 参数可为子元素实例或子元素 ID		
<code>move()</code>	<code>`(target_container: Element</code> <code>None = None,</code> <code>target_index: int = -1,</code> <code>target_slot: str</code>	将当前行（或其子元素）移动到其他容器： - <code>target_container</code> ：目标容器（默认父容器） - <code>target_index</code> ：目标位置索引（-1 表示末尾） - <code>target_slot</code> ：目标容器的插槽（默认默认插槽）	<code>None = None) -> None`</code>	

(二) 样式与外观配置

方法名	签名	说明				
<code>default_classes()</code>	<code>`(add: str</code>	<code>None = None, remove: str</code>	<code>None = None, toggle: str</code>	<code>None = None, replace: str</code>	<code>None = None) -> type[Self]`</code>	批量修改默认 HTML 类: - <code>add</code> : 添加类 (空格分隔) - <code>remove</code> : 移除类- <code>toggle</code> : 切换类 (存在则移除, 不存在则添加) - <code>replace</code> : 替换所有类 (2.7.0 + 支持)
<code>default_props()</code>	<code>`(add: str</code>	<code>None = None, remove: str</code>	<code>None = None) -> type[Self]`</code>	批量修改默认 Quasar props: - <code>add</code> : 添加 props (如 "dense shadow") - <code>remove</code> : 移除 props		
<code>default_style()</code>	<code>`(add: str</code>	<code>None = None, remove: str</code>	<code>None = None, replace: str</code>	<code>None = None) -> type[Self]`</code>	批量修改默认 CSS 样式: - <code>add</code> : 添加样式 (分号分隔) - <code>remove</code> : 移除样式- <code>replace</code> : 替换所有样式	
<code>tooltip()</code>	<code>(text: str) -> self</code>	为行容器添加 tooltip (鼠标悬浮时显示的提示文本)				

(三) 数据绑定与可见性控制

方法名	签名	说明			
<code>bind_visibility()</code>	<code>`(target_object: Any, target_name: str = 'visible', forward: Callable[[Any], Any]</code>	<code>None = None, backward: Callable[[Any], Any]</code>	<code>None = None, value: Any = None, strict: bool</code>	<code>None = None) -> Self`</code>	双向绑定可见性: 将行的可见性与目标对象的属性关联 (支持正向 / 反向转换函数)

方法名	签名	说明			
<code>bind_visibility_from()</code>	<code>`(target_object: Any, target_name: str = 'visible', backward: Callable[[Any], Any]</code>	<code>None = None, value: Any = None, strict: bool</code>	<code>None = None) -> Self`</code>	单向绑定可见性（从目标对象到行）：目标属性变化时同步行的可见性	
<code>bind_visibility_to()</code>	<code>`(target_object: Any, target_name: str = 'visible', forward: Callable[[Any], Any]</code>	<code>None = None, strict: bool</code>	<code>None = None) -> Self`</code>	单向绑定可见性（从行到目标对象）：行的可见性变化时同步目标属性	
<code>set_visibility()</code>	<code>(visible: bool) -> None</code>	直接设置可见性（ <code>True</code> 显示， <code>False</code> 隐藏）			

(四) 事件与交互

- `on()` 方法**
 - 签名：`(type: str, handler: events.Handler[events.GenericEventArguments] | None = None, args: None | Sequence[str] | Sequence[Sequence[str] | None] = None, throttle: float = 0.0, leading_events: bool = True, trailing_events: bool = True, js_handler: str = '(...args) => emit(...args)') -> Self`
 - 说明：用于绑定事件，支持同时设置 Python 回调（服务端处理）和 JavaScript 回调（客户端处理）。可通过 `args` 参数过滤事件传递的参数，`throttle` 参数设置事件触发的最小时间间隔（单位：秒）以避免频繁触发，`leading_events` 控制是否在首次事件发生时立即触发处理器（默认 `True`），`trailing_events` 控制是否在最后一次事件发生后触发处理器（默认 `True`）。默认的 `js_handler` 会将所有参数转发给服务端，可自定义该函数实现客户端本地事件处理或参数转换。
 - 更新版本：2.18.0 及以上版本支持同时指定 Python handler 和 JS handler。
- `mark()` 方法**
 - 签名：`(*markers: str) -> Self`
 - 说明：用于为元素添加标记，标记可用于测试时的元素查询，也可用于减少全局变量依赖或简化依赖传递。传入的参数为一个或多个字符串（支持空格分隔的单个字符串），会替换元素已有的标记。
 - 更新版本：无特定更新版本，为基础内置方法。

(五) 资源与插槽管理

方法名	签名	说明	
<code>add_resource()</code>	<code>`(path: str</code> <code>Path) -> None`</code>		添加资源文件（如 CSS/JS 文件夹）：用于行容器专属的样式 / 脚本
<code>add_dynamic_resource()</code>	<code>(name: str,</code> <code>function:</code> <code>Callable) -> None</code>	添加动态资源：通过函数动态生成资源响应（如动态 CSS）	
<code>add_slot()</code>	<code>`(name: str,</code> <code>template: str</code> <code>None = None) -></code> <code>Slot`</code>		添加插槽：支持 Vue 插槽机制，用于复杂子元素嵌套（如自定义行头部 / 尾部）

(六) 其他通用方法

方法名	签名	说明
<code>delete()</code>	<code>() -> None</code>	删除当前行及所有子元素（彻底从 DOM 中移除）
<code>update()</code>	<code>() -> None</code>	同步更新元素到客户端：修改属性 / 样式后调用，确保界面实时刷新
<code>ancestors()</code>	<code>(include_self: bool = False) -></code> <code>Iterator[Element]</code>	迭代获取所有祖先元素 (<code>include_self=True</code> 包含自身)
<code>descendants()</code>	<code>(include_self: bool = False) -></code> <code>Iterator[Element]</code>	迭代获取所有子元素（递归， <code>include_self=True</code> 包含自身）
<code>get_computed_prop()</code>	<code>(prop_name: str, timeout: float</code> <code>= 1) -> AwaitableResponse</code>	获取计算属性（需异步等待，如获取实际渲染后的宽度 / 高度）
<code>run_method()</code>	<code>(name: str, *args: Any, timeout:</code> <code>float = 1) -> AwaitableResponse</code>	调用客户端方法（需异步等待，如执行 Vue 组件的方法）

五、典型使用场景示例

场景 1：基础水平布局（按钮组）

```
from nicegui import ui

with ui.row(align_items="center", style="gap: 8px; margin: 16px 0;"):
    ui.button("新增", icon="add")
    ui.button("编辑", icon="edit")
    ui.button("删除", icon="delete", color="red")
```

```
ui.run()
```

场景 2：动态添加 / 删除子元素

```
from nicegui import ui

row = ui.row(style="gap: 8px;")
count = 0

def add_item():
    global count
    count += 1
    # 新增子元素并添加删除按钮
    with row:
        ui.label(f"项目 {count}")
        ui.button("x", size="sm", on_click=lambda e:
e.sender.parent.remove(e.sender.parent.children[-2:]))

ui.button("添加项目", on_click=add_item)
ui.button("清空所有", on_click=row.clear)

ui.run()
```

场景 3：可见性双向绑定

```
from nicegui import ui

class AppState:
    def __init__(self):
        self.show_row = False

state = AppState()

# 绑定行的可见性到 state.show_row (双向)
with ui.row().bind_visibility(state, 'show_row'):
    ui.label("绑定状态的行")
    ui.button("测试按钮")

# 切换状态的复选框 (同样绑定到 state.show_row)
ui.checkbox("显示行", value=False).bind_value(state, 'show_row')

ui.run()
```

场景 4：自定义样式与插槽

```
from nicegui import ui

# 自定义行样式 (使用 Tailwind 类)
row = ui.row(classes="bg-blue-50 rounded-xl p-6 gap-4", align_items="stretch")
```

```
# 普通子元素
with row:
    ui.card("卡片 1", classes="flex-1")
    ui.card("卡片 2", classes="flex-1")

# 添加自定义插槽（如右侧固定按钮）
slot = row.add_slot("end")
with slot:
    ui.button("更多", icon="more_vert", color="primary")

ui.run()
```

六、注意事项

- 换行行为**：默认 `wrap=True`，子元素宽度超出容器时自动折行；若需强制单行（溢出滚动），可设置 `wrap=False` 并配合 `style="overflow-x: auto;"`。
- 对齐方式**：`align_items` 仅控制垂直方向对齐，水平方向对齐需通过 `style`（如 `justify-content: space-between`）或 `classes` 实现。
- 样式优先级**：`style`（内联样式）优先级高于 `classes`（外部样式），`default_classes/default_style` 需在元素实例化前调用（用于全局默认样式）。
- 数据绑定**：`bind_visibility` 支持双向同步，初始值以目标对象属性为准；若需自定义转换逻辑，可通过 `forward/backward` 参数传入函数。
- 事件处理**：2.18.0 及以上版本支持同时设置 Python handler 和 JS handler，JS handler 可先处理事件再决定是否向服务端发送请求（减少网络开销）。
- 插槽使用**：`add_slot` 用于复杂布局（如行内分区域），普通子元素默认放入 `default` 插槽，嵌套时需注意插槽栈的切换。

七、总结

`ui.row` 是 NiceGUI 中功能强大的水平布局组件，通过初始化参数可快速配置布局行为，通过属性和方法支持动态样式调整、子元素操作、数据绑定等高级需求。其设计贴合 Vue 生态，支持插槽、Props、事件等特性，同时兼容 Tailwind/Quasar 样式体系，适用于从简单按钮组到复杂数据面板的各类水平布局场景。使用时需结合实际需求选择参数和方法，合理利用样式和绑定功能，可大幅提升界面开发效率和灵活性。